CS 305 Project One

Document Revision History

Version	Date	Author	Comments
1.0	9/16/2026	Jonathan Miranda	

Client

Developer

Jonathan Miranda

1. Interpreting Client Needs

Artemis Financial Application Security Modernization

Communications with the application should be secured using encryption-enabled protocols such as HTTPS and TLS. Failure to secure communications through encryption exposes information sent between the client and application to potential interception and modification by malicious parties.

The supplied application does not appear to include an internal HTTPS configuration. So long as the project specifications do not explicitly state that Artemis Financial is going to perform international transactions, it would not be wise to assume this, and further clarification should be sought from the client. Subsequently, if Artemis Financial does expand into other countries, the development team must determine the security and privacy requirements of each country in order to operate within them.

Furthermore, possible governmental restrictions on secure communication, such as the use of encryption between the client and web application or between the application and server, need to be checked for the client. These requirements may depend on the information processed by Artemis Financial, the geographical distribution of its customers, and where the Artemis Financial servers are located.

When interacting with the web application's REST API, there are many potential threats. A user could intentionally send malicious data to test for vulnerabilities, or a user could inadvertently provide incorrect or unwanted data. The service could also be affected by a denial-of-service (DoS) attack if an attacker intentionally sends too much data or otherwise consumes too many application resources.

Open-source libraries used in the application, such as Spring Boot, Spring Framework, Tomcat, SnakeYAML, Jackson, Bouncy Castle, and the logging libraries, have to be updated on a regular basis to reduce the risk of known security vulnerabilities remaining in the application. In addition to secure communication, input validation, secure database access, better error handling, and automated security testing within the build process should also be included in the application's security measures.

2. Areas of Security

The Vulnerability Assessment Process lists seven main security areas: input validation, APIs, cryptography, client/server communication, code error handling, code quality, and encapsulation. Based on the supplied application and source code, I reviewed these areas to determine which ones apply.

Input Validation

The application receives values directly from REST endpoints. In CRUDController.java, the business_name value is accepted without checking the format, size, or expected characters. Even though the supplied code does not currently use business_name in a database query, it should still be validated before it is passed into other application logic.

GreetingController.java also accepts a name value directly from a request. This value should also be validated before it is used.

APIs

Artemis Financial uses APIs. The application is designed to be RESTful, and the controllers expose endpoints for read access (/read) and greeting messages (/greeting). However, no authentication or authorization has been implemented for the API.

The supplied code does not demonstrate any authentication or authorization controls for the exposed API endpoints provided by the application controllers. Because of this, API security is an important area for Artemis Financial.

Client/Server

The web server is a Spring Boot application and connects to a MySQL database. Communication with the database should be done over secure connections.

All interactions with the database should be done using its own application account and granted only the minimum permissions needed for the application to function. Also, if MySQL has to be accessed remotely, the database connection should be encrypted.

Code Error Handling

It is also important to note that an SQLException is caught in the DocData.java class and e.printStackTrace() is called. This can leak internal information about the application.

Stack traces can contain class names, database configuration, and other internal information and should therefore be kept out of the output. It is better to log these errors than to print stack traces.

Code Quality

As for code quality, even though the application is running, there are still several issues. The code uses hard-coded credentials, contains incomplete methods, outdated dependencies, too little testing, and weak validation of financial calculations.

All of these issues could lead to reliability problems and create additional security issues as the application grows.

Encapsulation

The customer class does not declare the account_balance variable as private, meaning that other classes in the same package could change the balance of a customer unexpectedly.

The account information of a customer should be stored in private variables within the customer class and changed through methods that include validation and the related business logic.

Cryptography

Secure communication is an important requirement for Artemis Financial, but this was not selected as one of the major manual code-review areas for the supplied Java source. The application does use the Bouncy Castle cryptographic library; however, vulnerabilities in this library are better identified through a dependency scan rather than a manual review of the application code itself.

3. Manual Review

The following findings were identified by manually reviewing the supplied Java classes, Maven configuration, and application.properties file.

#	Area	Location	Finding / Risk
1	Client/Server	DocData.java	The database username and password are hard-coded as root/root. Anyone with access to the source code could obtain the credentials.
2	Least Privilege	DocData.java	The application connects to the database using the root account, which gives the application more privileges than it should need.

3	Client/Server	DocData.java	The JDBC connection does not explicitly require encrypted database transport. A remote database connection could expose traffic if TLS is not being enforced somewhere else.
4	Client/Server	application.properties	The server uses port 8081, but no SSL/TLS configuration is included. Unless HTTPS is being provided externally, client traffic would be unencrypted.
5	Code Error	DocData.java	SQLException handling calls e.printStackTrace(), which could expose application and database details.
6	APIs	CRUDController.java / GreetingController.java	No authentication or authorization controls are visible for the REST endpoints.
7	Input Validation	CRUDController.java	business_name is accepted from a request without validation or size restrictions.
8	Availability	application.properties	Multipart uploads allow files up to 200 MB and requests up to 215 MB. If an upload endpoint is exposed, large requests could consume application resources.

9	Encapsulation	customer.java	account_balance is not private, allowing other code in the package to modify the balance directly.
10	Input Validation / Code Quality	customer.java	deposit(int a) accepts any integer. A negative value could lower the balance, while a very large value could cause integer overflow.

I would not report SQL injection as a confirmed vulnerability based on the code provided above. The DocData.read_document() method accepts key and value parameters; however, it does not execute any SQL statements using these provided values.

This code alone does not provide enough evidence to report SQL injection as a confirmed vulnerability. However, if these values are later used in database queries, they should be used with parameterized queries.

In addition to these possible security issues, there are also a number of issues related to code quality. In myDateTime.java, the accessor methods do not appear to store the supplied values. The supplied test class also contains only the contextLoads() test, which does not provide much testing of the application.

Additional functional testing and security testing would be needed before determining whether the application is ready to go into production.

The findings above meet the Project One requirement to identify at least seven to ten vulnerabilities and include the location of each finding.

4. Static Testing

Here you can see the output from OWASP Dependency-Check version 12.2.2 when run on the Artemis Financial Maven project. Dependencies scanned totaled 38, with 20 unique dependencies identified.

In total, 11 out of the 38 dependencies included in the scan were vulnerable. The total number of findings was 252, of which 0 were suppressed.

Project: rest-service

com.twk:rest-service:0.0.1-SNAPSHOT

Scan Information (show less):

- *dependency-check version*: 12.2.2
- *Report Generated On*: Wed, 16 Sep 2026 19:29:40 -0400
- *Dependencies Scanned*: 38 (20 unique)
- *Vulnerable Dependencies*: 11
- *Vulnerabilities Found*: 252
- *Vulnerabilities Suppressed*: 0
- *NVD API Last Checked*: 2026-09-16T19:26:14-04
- *NVD API Last Modified*: 2026-09-16T23:16:54Z

Note that an asterisk (*) is added to Critical dependencies that contain a known exploited vulnerability. There are 252 findings, so listing all of the CVEs would result in a report that is far too long for this assessment.

The full HTML report has been retained as evidence. Below are the main vulnerable components that were found and a selection of the findings for each.

The report contains CVE information about the identified vulnerabilities, as well as descriptions and references to the National Vulnerability Database (NVD), CISA, and vendor security advisories.

Dependency	Highest Severity	CVE Count
bcprov-jdk15on-1.46	High	4

hibernate-validator-6.0.18.Final	Medium	3
jackson-databind-2.10.2	High	11
log4j-api-2.12.1	Medium	3
logback-core-1.2.3	High	2
snakeyaml-1.25	Critical	8
spring-boot-2.2.4.RELEASE	Critical	9
spring-core-5.2.3.RELEASE	Critical*	29
spring-web-5.2.3.RELEASE	Critical*	30
tomcat-embed-core-9.0.30	Critical*	76
tomcat-embed-websocket-9.0.30	Critical*	77

Bouncy Castle 1.46

Bouncy Castle 1.46 has four vulnerabilities identified in the report, with the highest rated as High.

One example is CVE-2016-1000338, which involves improper validation of ASN.1 encoding during DSA signature verification and has a CVSS version 3 score of 7.5.

The recommended solution is to upgrade Bouncy Castle to a version outside the affected range.

Hibernate Validator 6.0.18.Final

This vulnerability affects Hibernate Validator 6.0.18.Final with ID CVE-2025-35036. The root cause of the vulnerability is how Expression Language is processed for validation messages that contain user-controlled input.

Later versions of Hibernate Validator changed this behavior. User-controlled data should not be placed directly into constraint violation messages, and the library should be upgraded to a supported version.

Jackson Databind 2.10.2

Jackson Databind 2.10.2 has eleven vulnerability findings.

One of these is CVE-2026-54512, which has a CVSS score of 8.1 and involves polymorphic deserialization. The report lists corrected versions including 2.18.8, 2.21.4, and 3.1.4.

There are also older findings for Jackson Databind, such as CVE-2020-25649, which affects XML External Entities (XXE).

Log4j API 2.12.1 and Logback Core 1.2.3

The report lists multiple findings for Log4j API 2.12.1 and Logback Core 1.2.3. Whether all of these vulnerabilities can actually be exploited depends on the logging configuration used by the application.

Since logging libraries are used throughout the application, it would probably be more efficient to update these as part of the overall framework upgrade.

SnakeYAML 1.25

SnakeYAML 1.25 has eight vulnerability findings, with the highest severity being Critical.

With a CVSS score of 9.8, CVE-2022-1471 allows unsafe classes to be created when YAML from an untrusted source is deserialized. This could potentially allow remote code execution.

The report recommends using SafeConstructor when deserializing YAML from unknown sources. It also recommends upgrading to version 2.0 or later.

Spring Boot 2.2.4.RELEASE

Spring Boot 2.2.4.RELEASE has nine findings.

One example is CVE-2022-27772, which involves temporary-directory hijacking in Spring Boot versions before 2.2.11.

These dependencies should be updated as part of an upgrade of the Spring Boot baseline. Spring Boot manages many of the transitive dependencies in the project, so upgrading Spring Boot could also resolve many of the dependency issues found in the report.

Spring Core and Spring Web 5.2.3.RELEASE

Dependency-Check found 29 findings for Spring Core and 30 for Spring Web 5.2.3.RELEASE. Both are marked as containing a known exploited vulnerability.

One example is CVE-2022-22965, which could permit remote code execution through data binding in certain Spring MVC or Spring WebFlux applications.

Since this project is configured to use Java 8, I would need to check the JDK version actually being used at runtime before concluding that this vulnerability can be exploited.

Even if this specific vulnerability does not apply, both dependencies are outdated and should still be upgraded.

Apache Tomcat 9.0.30

In addition to the 77 vulnerabilities listed for the tomcat-embed-websocket dependency, Dependency-Check also found 76 vulnerabilities in tomcat-embed-core.

CVE-2020-1938 is identified by Dependency-Check as a CISA Known Exploited Vulnerability involving the Apache JServ Protocol (AJP).

Whether this vulnerability can actually be exploited depends on several factors, including whether AJP is enabled, whether it can be reached by an attacker, and how Tomcat is configured.

CVE-2025-24813 is also listed as a CISA Known Exploited Vulnerability affecting Apache Tomcat. According to the report, this vulnerability could potentially allow remote code execution, information disclosure, or malicious content injection under certain configurations.

It is recommended to upgrade the Tomcat components through a supported Spring Boot dependency baseline. The Tomcat configuration should also be reviewed to make sure it does not introduce additional vulnerabilities.

Static Test Interpretation

Note that there might be false positives as well as false negatives, so all findings have to be checked to determine whether they really apply to the web application.

The report listing 252 findings does not necessarily mean that there are 252 vulnerabilities that can be used to attack the Artemis Financial application. Some of the findings may not even apply to the application's current configuration.

Even considering the limitations of the scan, the results have exposed an old dependency baseline containing a high number of known vulnerabilities. Because of this, dependency scanning should become part of the normal software development process.

5. Mitigation Plan

A manual review of the web application's source code and a static security test of the Artemis Financial application have revealed the main security issues that need to be addressed. Some of these issues were found in the code written for the application, and others were found in the third-party components that the application uses.

The security problems fall into two categories: problems with the application code and configuration, and vulnerabilities in third-party dependencies used by the application.

An action list for these findings is below.

Priority	Finding	Recommended Action
1	Hard-coded database credentials	Remove root/root from the source code and store credentials securely in configuration files, environment variables, or a secrets-management system.
2	Excessive database privileges	Create a dedicated database account for the application with only the permissions it needs.
3	Unsecured communications	Require HTTPS/TLS for client traffic and encrypted connections to the database if it is accessed over a network.
4	Missing authentication / authorization	Implement user authentication for the REST application and enforce authorization on API endpoints.

5	Missing input validation	Validate incoming request parameters and financial values for the correct format, range, and size before processing.
6	Unsafe error handling	Replace printStackTrace() with proper server-side logging and return generic error messages to clients.
7	Financial model / encapsulation	Make financial fields private, validate changes through methods, reject negative deposits, prevent overflow, and use BigDecimal for decimal monetary values.
8	Large upload limits	Reduce the 200 MB file and 215 MB request limits to the actual business requirements and validate uploaded file types and sizes.
9	Outdated framework dependencies	Upgrade Spring Boot to a supported baseline so the underlying Spring Framework, Tomcat, Jackson, SnakeYAML, and logging libraries can also be upgraded.
10	Bouncy Castle 1.46	Upgrade Bouncy Castle to a supported version or remove it if the application does not need it.
11	Unsafe YAML deserialization	Upgrade SnakeYAML and use safe construction practices when parsing untrusted YAML data.
12	Continuous vulnerability detection	Keep OWASP Dependency-Check in the Maven build and run it regularly, ideally as part of a CI/CD pipeline, to detect new vulnerabilities.
13	Configuration-specific CVEs	Check the actual runtime configuration for findings such as Spring CVE-2022-22965 and Tomcat CVE-2020-1938 to

		determine whether they are exploitable or false positives before suppressing them.
14	Limited security testing	Add automated security tests covering authentication, authorization, input validation, invalid financial transactions, error handling, and oversized requests.

First, I would start by fixing the exposed database credentials and the excessive database permissions, followed by fixing the insecure communications and the missing application authentication and authorization.

I would deal with these issues first because they were found during the manual review of the source code, so I do not have to wait for the results of the dependency scan.

I would then update the Spring Boot Maven dependency to a supported newer version and re-run the report to see which of the originally identified transitive dependency vulnerabilities have been fixed and which still remain.

I would run Dependency-Check against the updated application to determine whether any of the previously reported vulnerabilities were fixed by the Spring Boot upgrade. I would also be able to identify any other open-source components within the Artemis Financial application that may still contain security vulnerabilities.

For vulnerabilities found in the dependencies of the Artemis Financial application that still need to be fixed, I would apply the necessary changes and run Dependency-Check again to see if any of the previously reported findings have gone away. Some of the issues may no longer be present, so I would have to verify whether the issue was actually fixed. Other issues may still be

present and would need to be reviewed to determine whether they still apply to the application or whether they are false positives.

Security scanning is not a one-off activity. New vulnerabilities are constantly discovered, which is why tools like Dependency-Check should be used as part of the normal development cycle of a project.

The purpose of this project is to highlight the security vulnerabilities that were found and explain how they can be fixed, not to actually fix them as part of the project.